Programming and Laboratory — Mod. 2

2. Tools

Giulio Ermanno Pibiri — giulioermanno.pibiri@unive.it
Department of Environmental Sciences, Informatics and Statistics
Ca’ Foscari University of Venice

Overview

- C++ project directory layout
- `make` and Makefiles
- Setting up a project step-by-step
- Debugging:
  - asserts
  - warnings
  - sanitizers

C++ project directory layout

- A C++ project is a collection of header and source files.

- Header files:
  - define class interfaces and function signatures
  - **`hpp`** extension
  - put into **`include/`**

- Source files:
  - contain the implementation of the signatures defined in the corresponding header files
  - **`cpp`** extension
  - put into **`src/`**

- Other sub-directories are **`test`** and **`external`**, for example.

```
parent_project_folder
├── build
├── include
├── src
└── tools
```

Our first example

- We will use this simple project as an example.

- The function's prototype is defined in the header file `include/myfunction.hpp`:

```
int myfunction(int x, double y);
```

- Header files are then included in the source files that need to use those functions. For example the file `src/myfunction.cpp` will begin with

```
#include "myfunction.hpp"
```

which is exactly the same as writing

```
int myfunction(int x, double y);
```

directly in the `.cpp` file.

```
myfunction-example-project
├── Makefile
├── build
├── include
│   └── myfunction.hpp
├── src
│   └── myfunction.cpp
└── tools
    └── main.cpp
```

Step by step using the terminal

1. `mkdir myfunction-example-project; cd myfunction-example-project`
2. `mkdir include src tools build`
3. (create and save the files `src/myfunction.cpp`, `include/myfunction.hpp`, and `tools/main.cpp`)
4. `g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o`
5. `g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/main`
6. `./build/main`

Step by step using the terminal

1. `mkdir myfunction-example-project; cd myfunction-example-project`
2. `mkdir include src tools build`
3. (create and save the files `src/myfunction.cpp`, `include/myfunction.hpp`, and `tools/main.cpp`)
4. `g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o`
5. `g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/main`
6. `./build/main`

Q. What if we forget this?

Step by step using the terminal

1. `mkdir myfunction-example-project; cd myfunction-example-project`
2. `mkdir include src tools build`
3. (create and save the files `src/myfunction.cpp`, `include/myfunction.hpp`, and `tools/main.cpp`)
4. `g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o`
5. `g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/main`
6. `./build/main`

Q. What if we forget this?

Q. What if we forget this?

Step by step using the terminal

1. `mkdir myfunction-example-project; cd myfunction-example-project`
2. `mkdir include src tools build`
3. (create and save the files `src/myfunction.cpp`, `include/myfunction.hpp`, and `tools/main.cpp`)
4. `g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o`
5. `g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/main`
6. `./build/main`

Q. What if we forget this?

Q. What if we forget this?

Q. What if we forget this instead?

Step by step using the terminal

1. `mkdir myfunction-example-project; cd myfunction-example-project`
2. `mkdir include src tools build`
3. (create and save the files `src/myfunction.cpp`, `include/myfunction.hpp`, and `tools/main.cpp`)
4. `g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o`
5. `g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/main`
6. `./build/main`

Q. What if we forget this?

Q. What if we forget this?

Q. What if we forget this instead?

We can replace step **4.** and **5.** with a single command

```
g++ -std=c++11 src/myfunction.cpp tools/main.cpp -I include -o build/main
```

that does **compilation and linking together in one step**.

Note: the object files can be deleted after linking since the linker links all of them together into the executable.

File by file compilation vs. compilation+linking in one step

1. **Compiling file by file and linking them together afterwards**:

   - only specific files have to be recompiled when needed instead of all of them each time (reduced compilation costs).

   - think about a real-world project with dozens of files…

2. **Compilation+linking**:

   - one command only, but

   - everything is recompiled each time and re-linked together.

How to scale up to larger projects?

- Using the methodology *“step by step using the terminal”* is great for learning but very **slow** for large projects:
- Typing by hand every compilation command in the terminal is tedious and **error-prone**.
- We would like to automate the compilation+linking pipeline so that we don’t have to manually run each task every single time…

How to scale up to larger projects?

- Using the methodology *“step by step using the terminal”* is great for learning but very **slow** for large projects:
- Typing by hand every compilation command in the terminal is tedious and **error-prone**.
- We would like to automate the compilation+linking pipeline so that we don’t have to manually run each task every single time…
- To **automate** the compilation+linking pipeline, we can use a tool called **Make**.

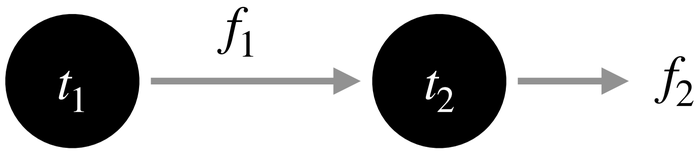
```
myfunction-example-project
├── Makefile
├── build      ???
├── include
│   └── myfunction.hpp
├── src
│   └── myfunction.cpp
└── tools
    └── main.cpp
```

Make

https://www.gnu.org/software/make/manual/make.html

- The program `make` can be used to **automate a pipeline** of tasks, each of which may depend on the output of other tasks.

- Example of pipeline: a task t_1 produces file f_1. Task t_2 requires f_1 and produces file f_2. Hence, task t_2 cannot be run if file f_1 has not been created (i.e. if task t_1 is not run first).

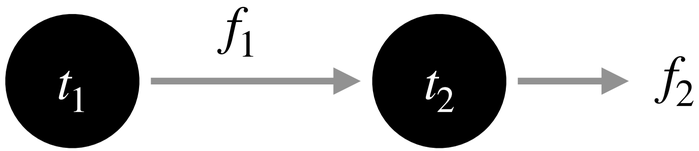


Make

https://www.gnu.org/software/make/manual/make.html

- The program `make` can be used to **automate a pipeline** of tasks, each of which may depend on the output of other tasks.

- Example of pipeline: a task t_1 produces file f_1. Task t_2 requires f_1 and produces file f_2. Hence, task t_2 cannot be run if file f_1 has not been created (i.e. if task t_1 is not run first).



- Typically used to **compile code** (many intermediate object files). The pipeline is the sequence of compilation tasks.

- The pipeline is specified in a “special” file, called **Makefile.**

Makefiles

- A Makefile is a textual file composed by *rules* and *options*.
- Let us consider rules first and options later.

- A *rule* is made up of:
  1. **target**: name of a file we want to generate *or* a more generic name for the rule (e.g., `all`, `clean`, `debug`, `release`, etc.)
  2. **prerequisites**: file(s) needed as input to generate the target or name(s) of rules needed to be executed before
  3. **recipe**: an action to be performed (e.g., one or more compilation commands)

- File structure:

```
<target>: <prerequisites>
<TAB><recipe>
```

A TAB character (‘`\t`’) is required here!

Let’s add a Makefile to our example project

Here is the basic file:

```
build/myfunction.o: src/myfunction.cpp include/myfunction.hpp
	g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o

build/mytool: build/myfunction.o tools/main.cpp
	g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/mytool

```

Let’s add a Makefile to our example project

Here is the basic file:

```
build/myfunction.o: src/myfunction.cpp include/myfunction.hpp
	g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o

build/mytool: build/myfunction.o tools/main.cpp
	g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/mytool

```

Q. What happens now if you type `make` from the project’s parent directory?

Let’s add a Makefile to our example project

Here is the basic file:

```
build/myfunction.o: src/myfunction.cpp include/myfunction.hpp
	g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o

build/mytool: build/myfunction.o tools/main.cpp
	g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/mytool

```

Q. What happens now if you type `make` from the project’s parent directory?

A. Only the first rule (`build/myfunction.o`) is executed.
We can then type `make build/mytool` to execute the second.
Or, even better, add a new first rule `all: build/mytool`.

Let’s add a Makefile to our example project

A refined version:

```
all: build/mytool

build/myfunction.o: src/myfunction.cpp include/myfunction.hpp
	g++ -std=c++11 -c src/myfunction.cpp -I include -o build/myfunction.o

build/mytool: build/myfunction.o tools/main.cpp
	g++ -std=c++11 build/myfunction.o tools/main.cpp -I include -o build/mytool
```

Q. What does this do instead?

So, in general…

- Behaviour of `make <target>`:
  1. If any prerequisite *changed* (timestamps are used) execute *recursively* the recipe of the prerequisite.
  2. Execute the recipe of the target.
- If no target specified (you just type `make`): execute the first rule (and only the first one).

So, in general…

- Behaviour of `make <target>`:
  1. If any prerequisite *changed* (timestamps are used) execute *recursively* the recipe of the prerequisite.
  2. Execute the recipe of the target.
- If no target specified (you just type `make`): execute the first rule (and only the first one).

FOLDERS

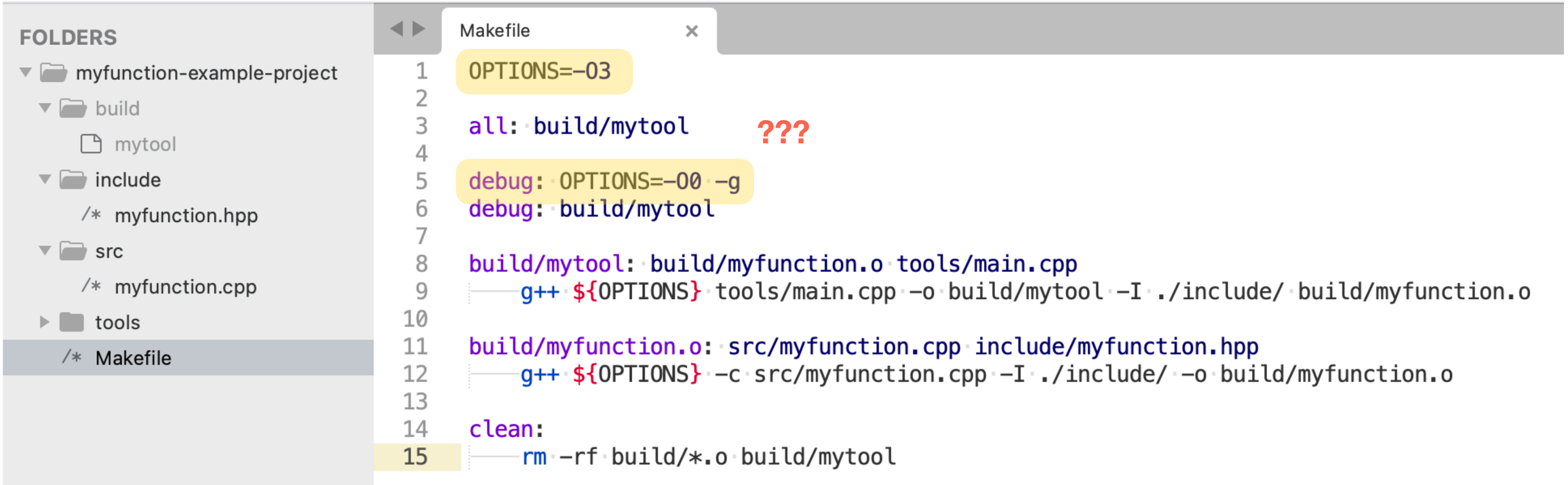
- myfunction-example-project
  - build
    - mytool
  - include
    - myfunction.hpp
  - src
    - myfunction.cpp
  - tools
  - Makefile

Makefile

```
OPTIONS=-O3

all: build/mytool

debug: OPTIONS=-O0 -g
debug: build/mytool

build/mytool: build/myfunction.o tools/main.cpp
	g++ ${OPTIONS} tools/main.cpp -o build/mytool -I ./include/ build/myfunction.o

build/myfunction.o: src/myfunction.cpp include/myfunction.hpp
	g++ ${OPTIONS} -c src/myfunction.cpp -I ./include/ -o build/myfunction.o

clean:
	rm -rf build/*.o build/mytool
```

So, in general…

- Behaviour of `make <target>`:
  1. If any prerequisite *changed* (timestamps are used) execute *recursively* the recipe of the prerequisite.
  2. Execute the recipe of the target.
- If no target specified (you just type `make`): execute the first rule (and only the first one).



Variables and options

FOLDERS

- myfunction-example-project
  - build
    - mytool
  - include
    - myfunction.hpp
  - src
    - myfunction.cpp
  - tools
  - Makefile

Makefile

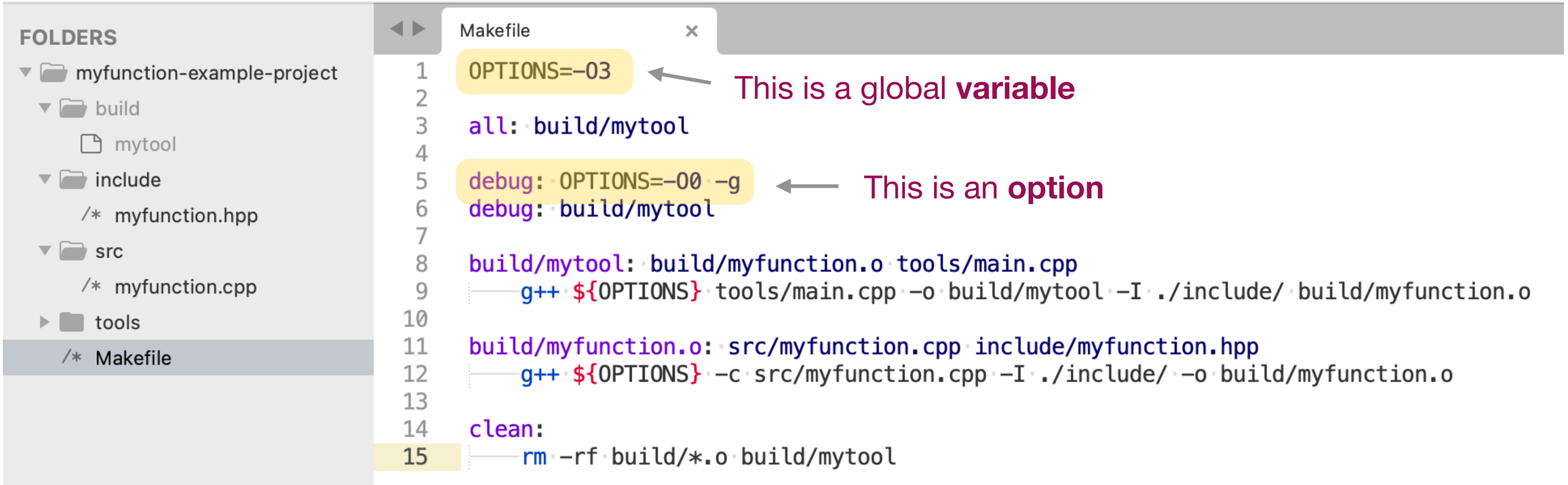
```
OPTIONS=-O3

all: build/mytool

debug: OPTIONS=-O0 -g
debug: build/mytool

build/mytool: build/myfunction.o tools/main.cpp
	g++ ${OPTIONS} tools/main.cpp -o build/mytool -I ./include/ build/myfunction.o

build/myfunction.o: src/myfunction.cpp include/myfunction.hpp
	g++ ${OPTIONS} -c src/myfunction.cpp -I ./include/ -o build/myfunction.o

clean:
	rm -rf build/*.o build/mytool
```

This is a global **variable**

This is an **option**

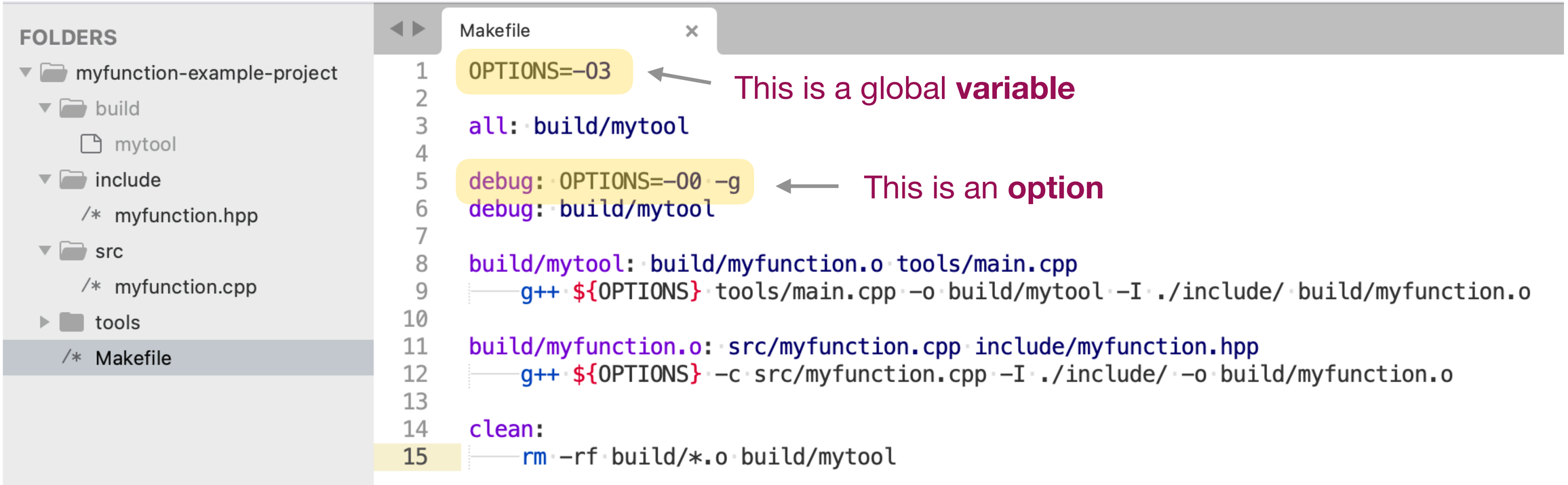
Variables and options

- A **variable** is just a placeholder for some text. If you write `VAR=T`, then `make` replaces the text `T` whenever it finds the name `VAR`. (To make the substitution, we must use the syntax `${VAR}` in the Makefile.)



Variables and options

- A **variable** is just a placeholder for some text. If you write `VAR=T`, then `make` replaces the text `T` whenever it finds the name `VAR`. (To make the substitution, we must use the syntax `${VAR}` in the Makefile.)
- An **option** is composed by the name of a **target** followed by a **variable assignment** (`make` looks for the token = after the target name) that defines target-specific options. If we have a target called X in our makefile and option X : `VAR=T`, the variable `VAR` will have value T when executing the rules for target X.



Makefiles

Not just for compiling code!

- **Q.** What does the following Makefile do?

```
final.txt: text1.txt text2.txt
	cat text1.txt text2.txt > final.txt

text1.txt:
	echo "text 1" > text1.txt

text2.txt:
	echo "text 2" > text2.txt
```

Makefiles

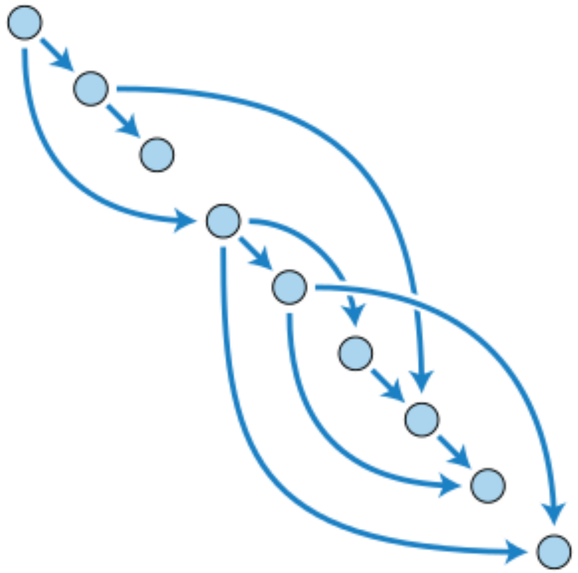
Not just for compiling code!

- **Q.** What does the following Makefile do?

```
final.txt: text1.txt text2.txt
	cat text1.txt text2.txt > final.txt

text1.txt:
	echo "text 1" > text1.txt

text2.txt:
	echo "text 2" > text2.txt
```

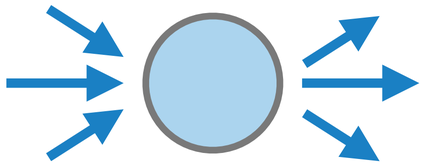
- **Q.** What happens if I run `make` two times?
- **Q.** What happens if I delete `text1.txt` and then run `make`?
- **Q.** What if I just "touch" the file `text1.txt`?
  Use the command `touch text1.txt` which changes the timestamp of `text1.txt`.

Makefiles as directed acyclic graphs (DAGs)



- The dependencies in Makefiles can be specified as **directed acyclic graphs** (or DAGs).
- Basic model:

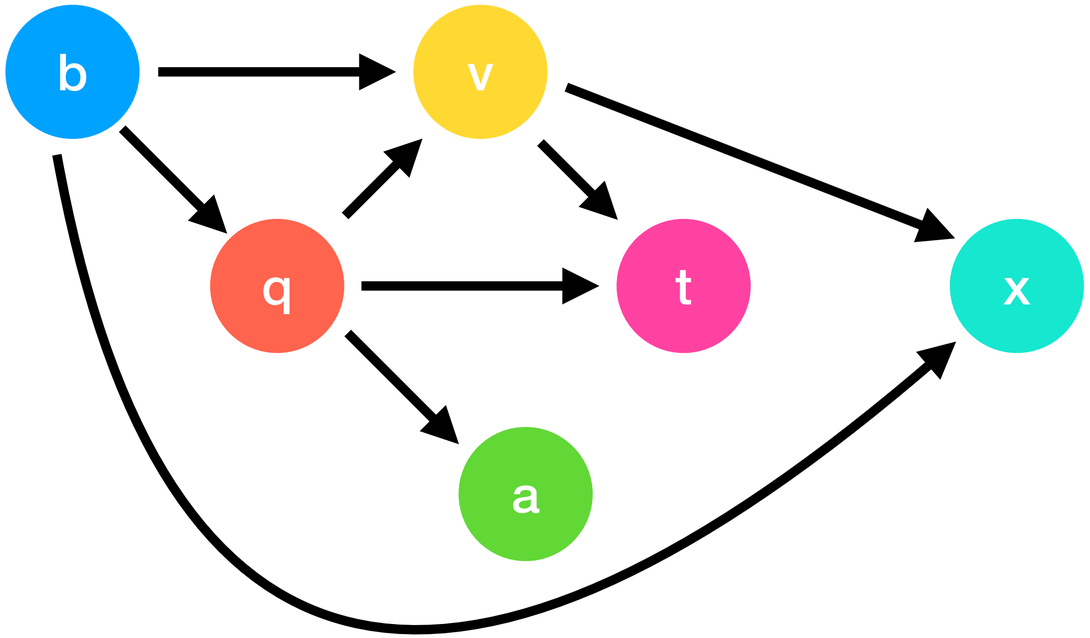
incoming edges:
prerequisites



outgoing edges:
results

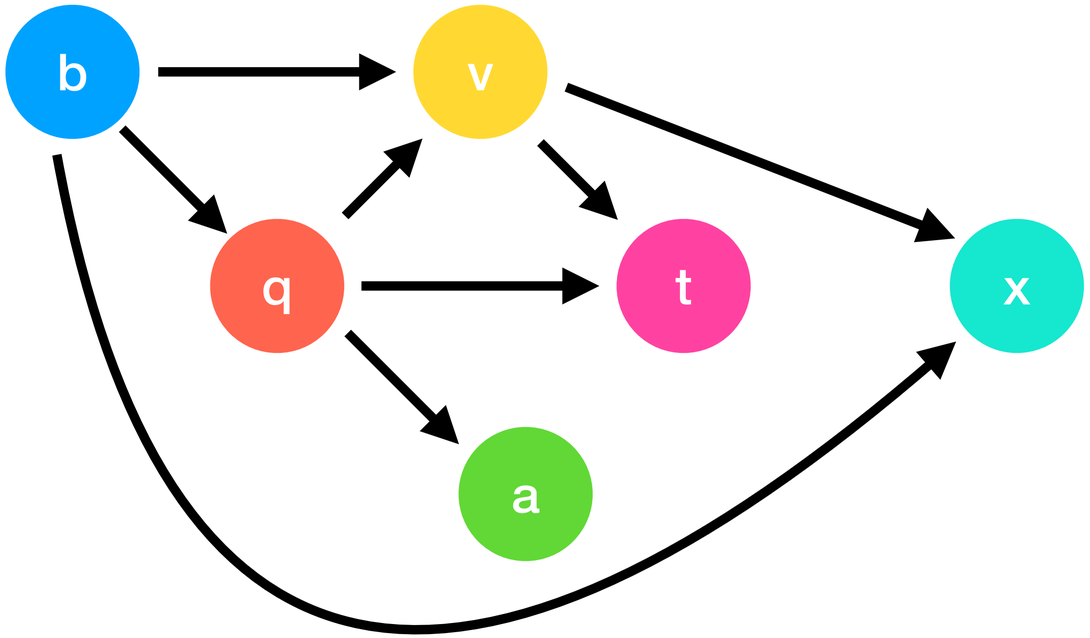
- To determine the order of execution of the targets, `make` sorts this graph so that arrows (dependencies) only go in one direction.
- This is called **topological sorting** (more details in CT0371) and is always possible on **acyclic** graphs.

DAG example

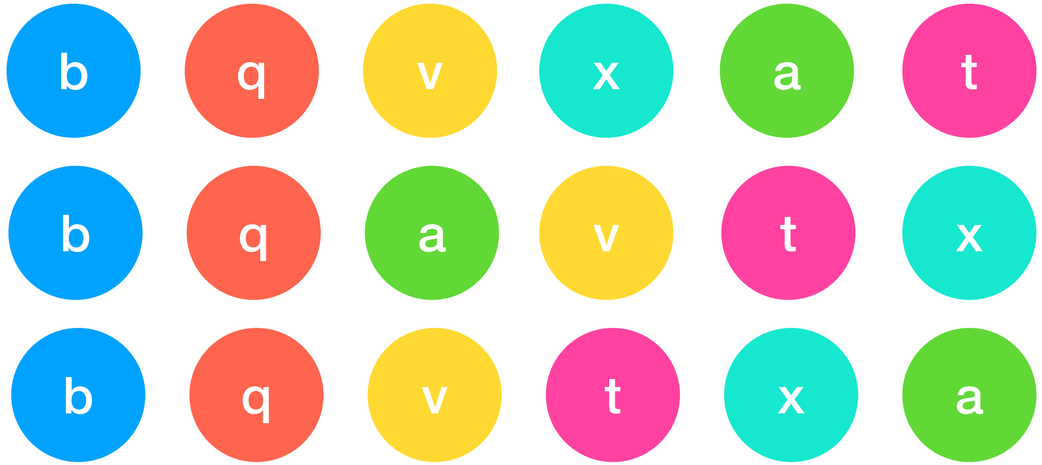


- Possible orders:

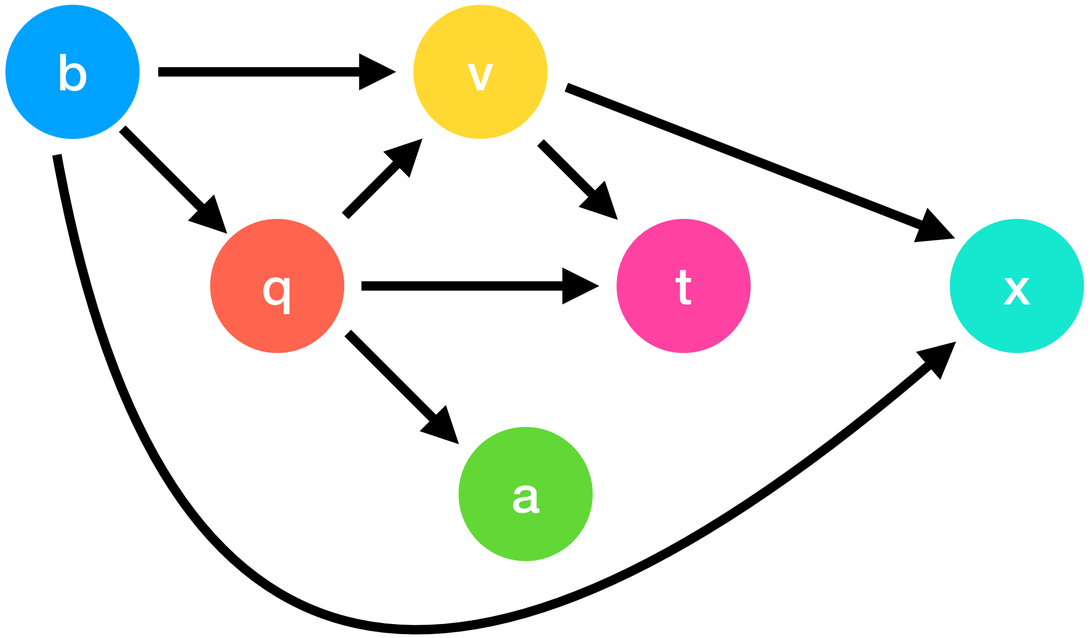
DAG example



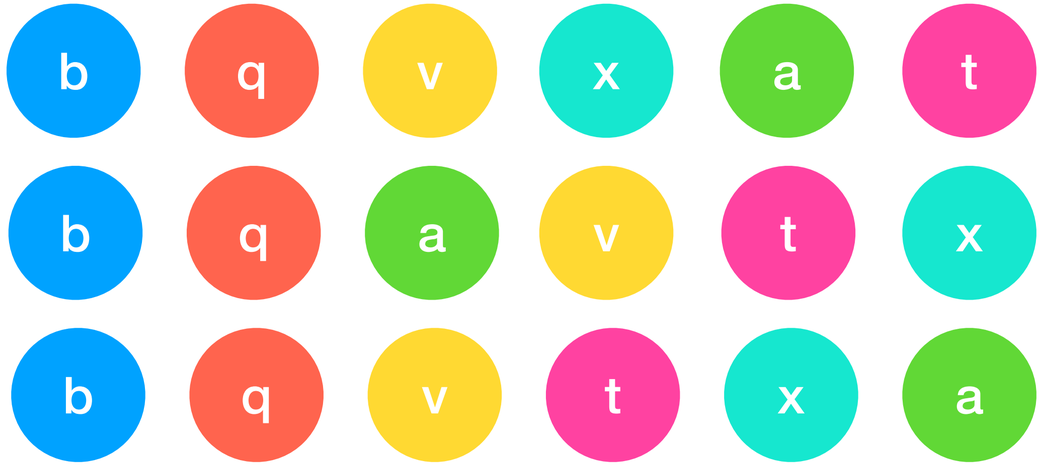
- Possible orders:



DAG example



- Possible orders:



Q. Can you describe an algorithm that outputs one of these orders?

Exercise

- Consider the following Makefile and draw the corresponding DAG.

```
project: main.o utils.o command.o
    g++ main.o utils.o command.o -o project

main.o: main.cpp defs.hpp
    g++ -c main.cpp

utils.o: utils.cpp defs.hpp command.hpp
    g++ -c utils.cpp

command.o: command.cpp defs.hpp command.hpp
    g++ -c command.cpp
```

Setting-up a project step-by-step

An example step-by-step

1. Using terminal, create a folder `power-root-example-project/` and enter it.
2. Create the project directory layout (sub-directories: `include/`, `src/`, `tools/`, `build/`).
3. Create a header `include/myfunctions.hpp` declaring the prototypes of these functions:

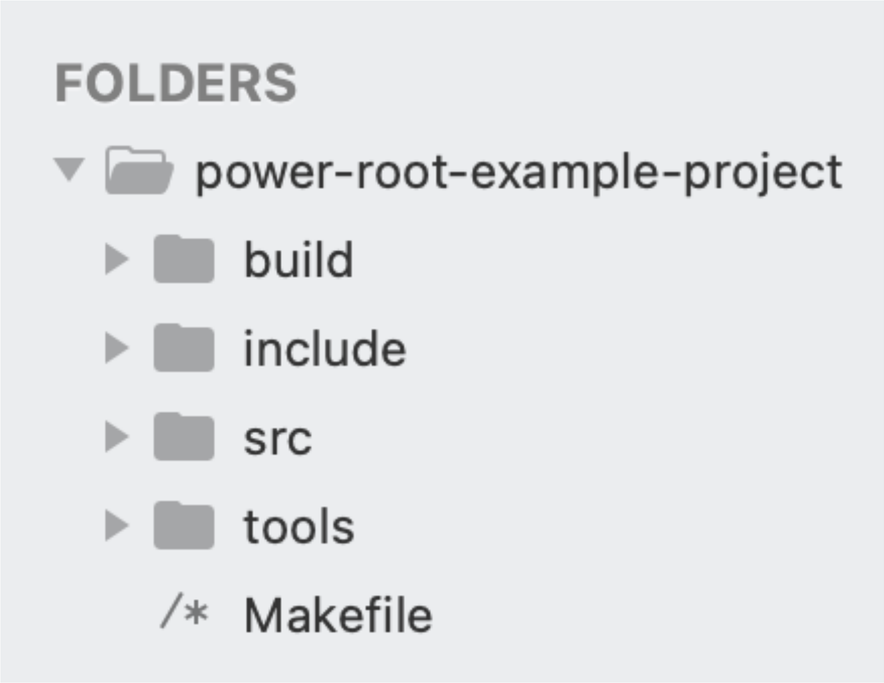
```
int power(int, int);
int root(int, int);
```

4. Create a source code file `src/power.cpp` defining the body of `int power(int x, int y)`, which returns x^y (use a `for` loop).
5. Create a source code file `src/root.cpp` defining the body of `int root(int x, int y)` which returns the largest integer z such that $z^y \leq x$ (use function `power` and a `while` loop).
6. Create a source code file `tools/main.cpp` containing the main function which computes the 3-rd root of 74088. The result has to be printed on the screen as `"The answer is"` followed by the result of `root(74088,3)`.
7. Create a Makefile that compiles everything in an executable called `theanswer`:
   - Specify the path of the includes using option `-I`.
   - Use optimization level 3.
   - Create a separate object file and a separate Makefile's target for each `.cpp` file.
   - Add also a `clean` target that removes all object files.
   - Optional: add other targets (e.g., `debug`) to compile with different options.
8. Compile and run the project.

Step 1 and 2

- Using terminal, create a folder `power-root-example-project/` and enter it.
- Create the project directory layout (sub-directories: `include/`, `src/`, `tools/`, `build/`).

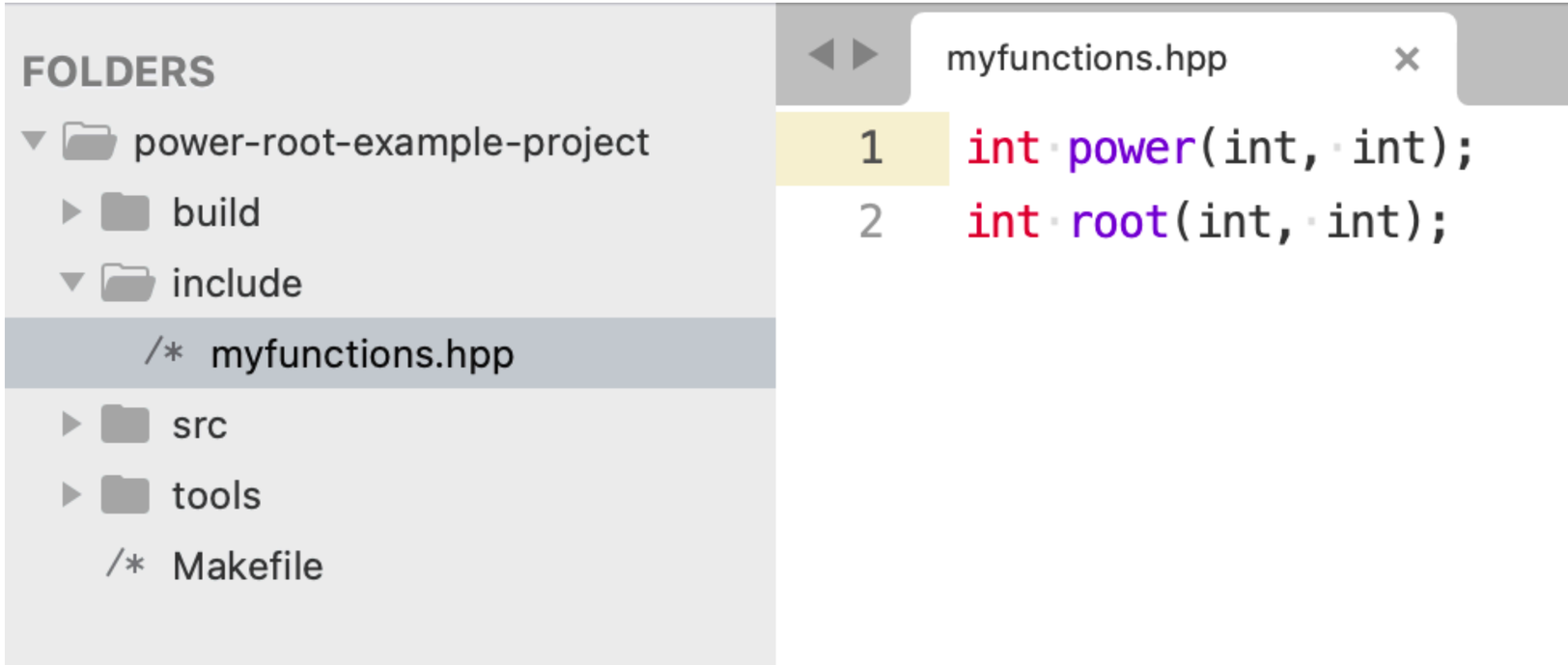
```
$ mkdir power-root-example-project; cd power-root-example-project
$ mkdir include src tools build
$ touch Makefile
```



Step 3

- Create a header `include/myfunctions.hpp` declaring the prototypes of these functions:

```
int power(int, int);
int root(int, int);
```



Step 4 and 5

- Create a source code file `src/power.cpp` defining the body of `int power(int x, int y)`, which returns x^y (just use a `for` loop).
- Create a source code file `src/root.cpp` defining the body of `int root(int x, int y)` which returns the largest integer z such that $z^y \leq x$ (use function `power` and a `while` loop).

FOLDERS

- power-root-example-project
  - build
  - include
    - myfunctions.hpp
  - src
    - power.cpp
    - root.cpp
  - tools
  - Makefile

power.cpp

```cpp
#include "myfunctions.hpp"

int power(int x, int y) {
    int result = 1;
    for (int i = 0; i < y; ++i) result *= x;
    return result;
}

```

root.cpp

```cpp
#include "myfunctions.hpp"

/* return the largest z such that z^y <= x */
int root(int x, int y) {
    int z = 0;
    while (power(z + 1, y) <= x) z++;
    return z;
}

```

Step 6

- Create a source code file `tools/main.cpp` containing the main function which computes the 3-rd root of 74088. The result has to be printed on the screen as `"The answer is"` followed by the result of `root(74088,3)`.

FOLDERS

- power-root-example-project
  - build
  - include
    - myfunctions.hpp
  - src
    - power.cpp
    - root.cpp
  - tools
    - main.cpp
  - Makefile

main.cpp

```cpp
#include <iostream>

#include "myfunctions.hpp"

int main() {
    int ans = root(74088, 3);
    std::cout << "The answer is " << ans << std::endl;
    return 0;
}

```

Step 7

- Create a Makefile that compiles everything in an executable called `theanswer`:
  - Specify the path of the includes using option `-I`.
  - Use optimization level 3.
  - Create a separate object file and a separate Makefile's target for each `.cpp` file.
  - Add also a `clean` target that removes all object files.
  - Optional: add other targets (e.g., `debug`) to compile with different options.

FOLDERS

- power-root-example-project
  - build
  - include
    - myfunctions.hpp
  - src
    - power.cpp
    - root.cpp
  - tools
  - Makefile

Makefile

```
OPTIONS=-O3

all: build/theanswer

debug: OPTIONS=-O0 -g
debug: build/theanswer

build/theanswer: build/root.o build/power.o tools/main.cpp
	g++ ${OPTIONS} tools/main.cpp -o build/theanswer -I include/ build/root.o build/power.o

build/root.o: src/root.cpp include/myfunctions.hpp
	g++ ${OPTIONS} -c src/root.cpp -o build/root.o -I include/

build/power.o: src/power.cpp include/myfunctions.hpp
	g++ ${OPTIONS} -c src/power.cpp -o build/power.o -I include/

clean:
	rm -rf build/*.o build/theanswer
```

Step 8

- Compile and run the project.
- From the parent project folder:

```
$ make
$ ./build/theanswer
```

or

```
$ make debug
$ ./build/theanswer
```

Debugging: asserts, warnings, and sanitizers

The “art” of debugging

What kind of errors can we run into when programming?

- **Linking-time** errors
- **Compilation-time** errors
- **Run-time** errors

The “art” of debugging

What kind of errors can we run into when programming?

- **Linking-time** errors
- **Compilation-time** errors
- **Run-time** errors

- Solutions for **run-time** errors:
  - debugging
  - pre/post conditions (asserts)
  - exceptions

The “art” of debugging

Debugging means finding **logical** errors in your program.

- Sometimes very hard to find: the program links, compiles, and runs but the **output is wrong**!
- Even worse, on some inputs the program works but on other input it does not work: hard to **reproduce** the bug.

  “*Reproducible bugs are candies.”* — cit.

- To find bugs, we need to inspect the program at **run-time** (while it is executing).

The “art” of debugging

Debugging means finding **logical** errors in your program.

- Sometimes very hard to find: the program links, compiles, and runs but the **output is wrong**!
- Even worse, on some inputs the program works but on other input it does not work: hard to **reproduce** the bug.

  “*Reproducible bugs are candies.”* — cit.

- To find bugs, we need to inspect the program at **run-time** (while it is executing).

- **Q.** How?
  - Plan you algorithms carefully!
  - Ensure pre/post conditions (asserts, see next).
  - Throw exceptions (next lectures).
  - Use a debugger (Linux has **gdb**, but we will not cover this).

Asserts — a minimal example

```
#include <cassert>

int main() {
    assert(3 != 3); /* throws a failed assertion */
}
```

- `assert` is a macro (with a functional form) that takes as input a boolean condition and halts the program if the condition is not satisfied.
- **The best way to document your code.**
- Use it to check pre/post conditions for your functions.

Asserts — a minimal example

```
#include <cassert>

int main() {
    assert(3 != 3); /* throws a failed assertion */
}
```

- `assert` is a macro (with a functional form) that takes as input a boolean condition and halts the program if the condition is not satisfied.
- **The best way to document your code.**
- Use it to check pre/post conditions for your functions.
- Compile with `g++ -DNDEBUG` (option `-D`, argument `NDEBUG`) to disable asserts (they are enabled by default): this way, they do not affect performance.
- Good idea to leave them enabled in debug mode, when developing a project.

Back to our example

FOLDERS
- power-root-example-project-with-asserts
  - build
  - include
    - myfunctions.hpp
  - src
    - power.cpp
    - root.cpp
  - tools
    - main.cpp
  - Makefile

myfunctions.hpp

```cpp
#include <cassert>

int power(int, int);
int root(int, int);
```

include the header “`cassert`”

use asserts to document your code

FOLDERS
- power-root-example-project-with-asserts
  - build
  - include
    - myfunctions.hpp
  - src
    - power.cpp
    - root.cpp
  - tools
    - main.cpp
  - Makefile

power.cpp

```cpp
#include "myfunctions.hpp"

int power(int x, int y) {
    assert(y >= 0);
    int result = 1;
    for (int i = 0; i < y; ++i) result *= x;
    return result;
}
```

main.cpp

```cpp
#include <iostream>

#include "myfunctions.hpp"

int main() {
    int ans = root(74088, 3);
    assert(ans == 42);
    std::cout << "The answer is " << ans << std::endl;
    return 0;
}
```

Back to our example

disable/enable asserts

FOLDERS

- power-root-example-project-with-asserts
  - build
  - include
    - myfunctions.hpp
  - src
    - power.cpp
    - root.cpp
  - tools
    - main.cpp
  - Makefile

Makefile

```
OPTIONS=-O3 -DNDEBUG -Wall -Wextra

all: build/theanswer

debug: OPTIONS=-O0 -g -DDEBUG -Wall -Wextra
debug: build/theanswer

build/theanswer: build/root.o build/power.o tools/main.cpp
	g++ ${OPTIONS} tools/main.cpp -o build/theanswer -I include/ build/root.o build/power.o

build/root.o: src/root.cpp include/myfunctions.hpp
	g++ ${OPTIONS} -c src/root.cpp -o build/root.o -I include/

build/power.o: src/power.cpp include/myfunctions.hpp
	g++ ${OPTIONS} -c src/power.cpp -o build/power.o -I include/

clean:
	rm -rf build/*.o build/theanswer
```

Compiler warnings

- The compiler can be instructed to report **warnings**.
- A warning is **not an error** but a suggestion that something is probably missing in your code or might go wrong…
- Comprehensive list of compiler flags to enable warnings: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html
- We will use two of them: `-Wall -Wextra`

Compiler warnings

- The compiler can be instructed to report **warnings**.
- A warning is **not an error** but a suggestion that something is probably missing in your code or might go wrong…
- Comprehensive list of compiler flags to enable warnings: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html
- We will use two of them: `-Wall -Wextra`

Memorize these lists! → `-Wextra`

```
-Wclobbered
-Wcast-function-type
-Wdeprecated-copy
-Wempty-body
-Wenum-conversion
-Wignored-qualifiers
-Wimplicit-fallthrough=3
-Wmissing-field-initializers
-Wmissing-parameter-type
-Wold-style-declaration
-Woverride-init
-Wsign-compare
-Wstring-compare
-Wredundant-move
-Wtype-limits
-Wuninitialized
-Wshift-negative-value
-Wunused-parameter
-Wunused-but-set-parameter
```

`-Wall`

```
-Waddress
-Warray-bounds=1
-Warray-compare
-Warray-parameter=2
-Wbool-compare
-Wbool-operation
-Wc++11-compat-Wc++14-compat
-Wcatch-value
-Wchar-subscripts
-Wcomment
-Wdangling-pointer=2
-Wduplicate-decl-specifier
-Wenum-compare
-Wenum-int-mismatch
-Wformat
-Wformat-overflow
-Wformat-truncation
-Wint-in-bool-context
-Wimplicit
-Wimplicit-int
-Wimplicit-function-declaration
-Winit-self
-Wlogical-not-parentheses
-Wmain
-Wmaybe-uninitialized
-Wmemset-elt-size
-Wmemset-transposed-args
-Wmisleading-indentation
-Wmismatched-dealloc
-Wmismatched-new-delete
-Wmissing-attributes
-Wmissing-braces
-Wmultistatement-macros
-Wnarrowing
-Wnonnull
-Wnonnull-compare
-Wopenmp-simd
-Wparentheses
-Wpessimizing-move
-Wpointer-sign
-Wrange-loop-construct
-Wreorder
-Wrestrict
-Wreturn-type
-Wself-move
-Wsequence-point
-Wsign-compare
-Wsizeof-array-div
-Wsizeof-pointer-div
-Wsizeof-pointer-memaccess
-Wstrict-aliasing
-Wstrict-overflow=1
-Wswitch
-Wtautological-compare
-Wtrigraphs
-Wuninitialized
-Wunknown-pragmas
-Wunused-function
-Wunused-label
-Wunused-value
-Wunused-variable
-Wuse-after-free=3
-Wvla-parameter
-Wvolatile-register-var
-Wzero-length-bounds
```

Compiler warnings — Example

```cpp
#include <iostream>

int main() {
  int x;
  int y = x * 7;
  std::cout << y << std::endl;
}
```

```
→  Desktop g++ test.cpp -o test -Wall -Wextra
test.cpp:5:11: warning: variable 'x' is uninitialized when used here [-Wuninitialized]
  int y = x * 7;
          ^
test.cpp:4:8: note: initialize the variable 'x' to silence this warning
  int x;
       ^
        = 0
1 warning generated.
→  Desktop
```

Always add these two flags in your Makefile

FOLDERS

- power-root-example-project-with-asserts
  - build
  - include
  - src
  - tools
  - Makefile

Makefile

```makefile
OPTIONS=-O3 -DNDEBUG -Wall -Wextra

all: build/theanswer

debug: OPTIONS=-O0 -g -DDEBUG -Wall -Wextra
debug: build/theanswer

build/theanswer: build/root.o build/power.o tools/main.cpp
	g++ ${OPTIONS} tools/main.cpp -o build/theanswer -I include/ build/root.o build/power.o

build/root.o: src/root.cpp
	g++ -c src/root.cpp -o build/root.o -I include/

build/power.o: src/power.cpp
	g++ -c src/power.cpp -o build/power.o -I include/

clean:
	rm -rf build/*.o build/theanswer
```

Memory debugging

- Most of C++ errors you will encounter in your code are related to (bad) **memory management**.
- For example, run-time errors often **abort** the program with (a rather uninformative):

  ```
  segmentation fault
  ```

- Often these are caused by:
  - **Out-of-bounds** memory accesses.
  - **Invalid** memory **allocation** requests.
  - **Memory leaks** — Incorrect memory handling could result in the RAM filling up quickly.

Memory debugging

- **IMPORTANT: the compiler does NOT detect these problems.**
- **Q.** Why?
  - Because memory monitoring can only be done at execution time: we can only detect problems while the program is running, **and** only if such problems occur.
  - A memory debugger **can** sometimes help… but it simulates a machine and annotate uninitialized/unallocated memory locations.

Memory debugging

- **IMPORTANT: the compiler does NOT detect these problems.**
- **Q.** Why?
  - Because memory monitoring can only be done at execution time: we can only detect problems while the program is running, **and** only if such problems occur.
  - A memory debugger **can** sometimes help… but it simulates a machine and annotate uninitialized/unallocated memory locations.
- For memory-related problems and more, we will use the compiler **sanitizers** — **compiler options** that add special instructions to your compiled code ("instrumentation") to check for illegal behaviour while the program is running.
- The are **four** main compiler sanitizers.

Sanitizer 1 — Address sanitizer (ASan)

- The most popular one. It detects memory errors that usually lead to segmentation faults.
- It correctly detects the errors like

  - out-of-bounds access (a.k.a, “buffer overflows” or “invalid reads”) and

  - use-after-free.
- Flag to use at compilation: `-fsanitize=address`
- Performance hit: $\approx 2\times$ slowdown.

Example — out-of-bounds access

FOLDERS

- sanitizers-example-project
  - build
  - tools
    - freed_memory_example.cpp
    - invalid_read_example.cpp
    - memory_leak_example.cpp
    - uninitialized_memory_example.
  - Makefile

invalid_read_example.cpp

```cpp
#include <iostream>

int main() {
  int *a = new int[10];
  std::cout << a[10] << std::endl;
  delete[] a;
  return 0;
}
```

```
==654763==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x604000000038 at pc 0x562d9e833322 bp 0x7ffe3a665210 sp 0x7ffe3a665200
READ of size 4 at 0x604000000038 thread T0
    #0 0x562d9e833321 in main tools/invalid_read_example.cpp:5
    #1 0x7f30277a6fcf in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #2 0x7f30277a707c in __libc_start_main_impl ../csu/libc-start.c:409
    #3 0x562d9e833204 in _start (/home/giulio/pel/slides/1-introduction/code/sanitizers-example-project/build/invalid_read_example+0x1204)

0x604000000038 is located 0 bytes to the right of 40-byte region [0x604000000010,0x604000000038)
allocated by thread T0 here:
    #0 0x7f3027d72337 in operator new[](unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:102
    #1 0x562d9e8332de in main tools/invalid_read_example.cpp:4
    #2 0x7f30277a6fcf in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
```

Example — access-after-free

FOLDERS
- sanitizers-example-project
  - build
  - tools
    - freed_memory_example.cpp
    - invalid_read_example.cpp
    - memory_leak_example.cpp
    - uninitialized_memory_example.
  - Makefile

freed_memory_example.cpp

```cpp
#include <iostream>

int main() {
  int *a = new int[10];
  delete[] a;
  std::cout << a[5] << std::endl;
  return 0;
}
```

```
==654728==ERROR: AddressSanitizer: heap-use-after-free on address 0x604000000024 at pc 0x55f6d70a6335 bp 0x7fff3f1671e0 sp 0x7fff3f1671d0
READ of size 4 at 0x604000000024 thread T0
    #0 0x55f6d70a6334 in main tools/freed_memory_example.cpp:6
    #1 0x7faeb0877fcf in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #2 0x7faeb087807c in __libc_start_main_impl ../csu/libc-start.c:409
    #3 0x55f6d70a6204 in _start (/home/giulio/pel/slides/1-introduction/code/sanitizers-example-project/build/freed_memory_example+0x1204)

0x604000000024 is located 20 bytes inside of 40-byte region [0x604000000010,0x604000000038)
freed by thread T0 here:
    #0 0x7faeb0e43e37 in operator delete[](void*) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:163
    #1 0x55f6d70a62f5 in main tools/freed_memory_example.cpp:5
    #2 0x7faeb0877fcf in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

previously allocated by thread T0 here:
    #0 0x7faeb0e43337 in operator new[](unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:102
    #1 0x55f6d70a62de in main tools/freed_memory_example.cpp:4
    #2 0x7faeb0877fcf in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
```

Sanitizer 2 — Undef. behaviour sanitizer (UBSan)

- C++ has many *“undefined” behaviours*. An undefined behaviour refers to code that is written in a way that the C++ standard simply does not define.

Sanitizer 2 — Undef. behaviour sanitizer (UBSan)

- C++ has many *“undefined” behaviours*. An undefined behaviour refers to code that is written in a way that the C++ standard simply does not define.

- Examples of undefined behaviour:

  - Accessing `A[10]` in a size-10 array `A`.
  - Dereferencing a `nullptr` (same as NULL pointer in C).
  - Accessing a memory area it’s been freed (Use-after-free).
  - Reading a local variable before assigning it a value.

- UBSan catches these logic errors that don't necessarily crash the program but make it unpredictable.

- Flag to use at compilation: `-fsanitize=undefined`

- Performance hit: very low.

Sanitizer 3 — Thread sanitizer (TSan)

- This sanitizer finds *data races* in multi-threaded code — bugs that are notoriously hard to reproduce because they depend on timing.
- Example: two threads accessing the same memory at the same time where at least one is a write access.
- Flag to use at compilation: `-fsanitize=thread`
- Performance hit: significant (e.g., $5 - 15\times$ slowdown).

Sanitizer 4 — Leak sanitizer (LSan)

- Focuses purely on memory that was allocated but never freed.
- **Note**: ASan usually includes LSan by default.
- Flag to use at compilation: `-fsanitize=leak`
- **Warning**: this compilation option on MacOS is not supported. On Linux, it works just fine.

Example — memory leak

FOLDERS

- sanitizers-example-project
  - build
  - tools
    - freed_memory_example.cpp
    - invalid_read_example.cpp
    - memory_leak_example.cpp
    - uninitialized_memory_example.
  - Makefile

memory_leak_example.cpp

```cpp
#include <iostream>

int main() {
  int *a = new int[10];
  return 0;
}
```

```
==657730==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 40 byte(s) in 1 object(s) allocated from:
    #0 0x7f9af29e6337 in operator new[](unsigned long) ../../../../src/libsanitizer/asan/asan_new_delete.cpp:102
    #1 0x56106d87225e in main tools/memory_leak_example.cpp:4
    #2 0x7f9af241afcf in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
```

Memory leaks

Gold rule to avoid memory leaks:

For each **new**, you have to write a corresponding **delete.**

In general: for each heap allocation, there should be a heap de-allocation (similarly, in C: for each `malloc`, you should have a `free`).

Memory leaks

- In general, don not call **`delete`** if you haven't called **`new`**.
- **Q.** Why is not (a manual) `delete` necessary in the following piece of code?

```
#include <vector>

int main() {

    auto v = std::vector<int>(100);

}
```

Memory leaks

- In general, don not call **`delete`** if you haven't called **`new`**.
- **Q.** Why is not (a manual) `delete` necessary in the following piece of code?

```
#include <vector>

int main() {

    auto v = std::vector<int>(100);

}
```

- **A.** Because the class destructor is automatically called when v goes **out of scope**.
- The new happens inside the class **constructor** and `delete` happens in the class **destructor** method.

Q. But why should you really care about memory bugs?

Q. But why should you really care about memory bugs?

A. For the final project!

You must use sanitisers when developing your code.

Back to our project

- Create a new target "`sanitizers`" in your Makefile with **at least these two flags**.
- You can enable/disable sanitizers at compile time:

```
$ make clean; make
$ make clean; make sanitizers
```

FOLDERS
- sanitizers-example-project
  - build
  - tools
    - freed_memory_example.cpp
    - invalid_read_example.cpp
    - memory_leak_example.cpp
    - uninitialized_memory_examp
  - Makefile

Makefile

```
all: OPTIONS=-O3 -DNDEBUG -Wall -Wextra
all: build/uninitialized_memory_example build/freed_memory_example build/invalid_read_exam

debug: OPTIONS=-O0 -g -DDEBUG -Wall -Wextra
debug: build/uninitialized_memory_example build/freed_memory_example build/invalid_read_ex

sanitizers: OPTIONS=-fsanitize=address -fno-omit-frame-pointer -g -DDEBUG -Wall -Wextra
sanitizers: build/uninitialized_memory_example build/freed_memory_example build/invalid_re

build/uninitialized_memory_example: tools/uninitialized_memory_example.cpp
	g++ -o build/uninitialized_memory_example ${OPTIONS} tools/uninitialized_memory_examp
```

Back to our project

- Create a new target "`sanitizers`" in your Makefile with **at least these two flags**.
- You can enable/disable sanitizers at compile time:

```
$ make clean; make
$ make clean; make sanitizers
```

what's this?

FOLDERS
- sanitizers-example-project
  - build
  - tools
    - freed_memory_example.cpp
    - invalid_read_example.cpp
    - memory_leak_example.cpp
    - uninitialized_memory_examp
  - Makefile

Makefile

```
all: OPTIONS=-O3 -DNDEBUG -Wall -Wextra
all: build/uninitialized_memory_example build/freed_memory_example build/invalid_read_exam

debug: OPTIONS=-O0 -g -DDEBUG -Wall -Wextra
debug: build/uninitialized_memory_example build/freed_memory_example build/invalid_read_ex

sanitizers: OPTIONS=-fsanitize=address -fno-omit-frame-pointer -g -DDEBUG -Wall -Wextra
sanitizers: build/uninitialized_memory_example build/freed_memory_example build/invalid_re

build/uninitialized_memory_example: tools/uninitialized_memory_example.cpp
	g++ -o build/uninitialized_memory_example ${OPTIONS} tools/uninitialized_memory_examp
```

Understanding the flag `no-omit-frame-pointer`

- First recall the role of the **stack**. The stack is a fixed-size memory area where your program keeps track of function calls, local variables, and “where to go back to” when a function finishes.

- Every time a function is called, a *frame* is created on the stack. CPUs use **two specific registers** to manage this:

Understanding the flag `no-omit-frame-pointer`

- First recall the role of the **stack**. The stack is a fixed-size memory area where your program keeps track of function calls, local variables, and “where to go back to” when a function finishes.

- Every time a function is called, a *frame* is created on the stack. CPUs use **two specific registers** to manage this:

  - **The stack pointer (SP):** Points to the very top (current end) of the stack. It moves constantly as you push and pop data.

Understanding the flag `no-omit-frame-pointer`

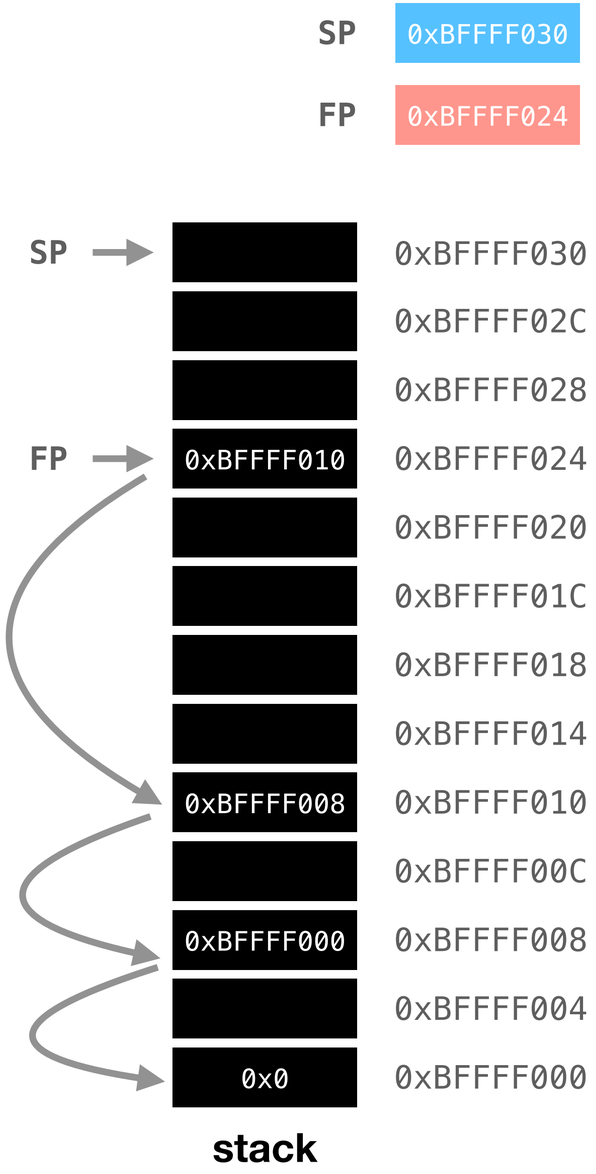
- First recall the role of the **stack**. The stack is a fixed-size memory area where your program keeps track of function calls, local variables, and “where to go back to” when a function finishes.

- Every time a function is called, a *frame* is created on the stack. CPUs use **two specific registers** to manage this:

  - **The stack pointer (SP):** Points to the very top (current end) of the stack. It moves constantly as you push and pop data.

  - **The frame pointer (FP):** Holds the address in memory of the beginning of the **current** stack frame.

    Each stack frame, instead, holds the address of the beginning of the **previous** stack frame.



Understanding the flag `no-omit-frame-pointer`

- By walking the frame pointers **backwards**, it is possible to reconstruct the stack trace of calls that lead to a specific problem.
- This is **very useful when debugging** to understand what is the path in your code that leads to the problem.

Understanding the flag `no-omit-frame-pointer`

- By walking the frame pointers **backwards**, it is possible to reconstruct the stack trace of calls that lead to a specific problem.
- This is **very useful when debugging** to understand what is the path in your code that leads to the problem.
- Without the frame pointer, the stack would just appear as a “list of numbers” with no clue of whether a number is the result of an expression or an address in memory.
- The flag `no-omit-frame-pointer` instructs the compiler not to delete the frame pointer.

An example

```
1   #include <iostream>
2
3   double g(double b) {
4       double* v = new double[5];
5       v[0] = 0;
6       for (int i = 1; i != 10; ++i) v[i] = v[i - 1] + b * i;
7       double ret = v[4];
8       delete[] v;
9       return ret;
10  }
11
12  double f(int a) {
13      static const double PI = 3.14;
14      double b = a * PI;
15      return g(b);
16  }
17
18  int main() {
19      int x = 3;
20      std::cout << f(x) << std::endl;
21      return 0;
22  }
```

Q. Where is the problem in this piece of code?

An example

```
1   #include <iostream>
2
3   double g(double b) {
4       double* v = new double[5];
5       v[0] = 0;
6       for (int i = 1; i != 10; ++i) v[i] = v[i - 1] + b * i;
7       double ret = v[4];
8       delete[] v;
9       return ret;
10  }
11
12  double f(int a) {
13      static const double PI = 3.14;
14      double b = a * PI;
15      return g(b);
16  }
17
18  int main() {
19      int x = 3;
20      std::cout << f(x) << std::endl;
21      return 0;
22  }
```

Q. Where is the problem in this piece of code?

An example

```cpp
#include <iostream>

double g(double b) {
    double* v = new double[5];
    v[0] = 0;
    for (int i = 1; i != 10; ++i) v[i] = v[i - 1] + b * i;
    double ret = v[4];
    delete[] v;
    return ret;
}

double f(int a) {
    static const double PI = 3.14;
    double b = a * PI;
    return g(b);
}

int main() {
    int x = 3;
    std::cout << f(x) << std::endl;
    return 0;
}
```

Q. Where is the problem in this piece of code?

With option `-fomit-frame-pointer`

```
[→  sanitizers-example-project git:(25-26) ✗ ./build/stack-trace
=================================================================
==4014==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x6040000003b
WRITE of size 8 at 0x6040000003b8 thread T0
    #0 0x00010042492c in g(double)+0x114 (stack-trace:arm64+0x10000092c)

0x6040000003b8 is located 0 bytes after 40-byte region [0x604000000390,0x60400
allocated by thread T0 here:
    #0 0x000100d87548 in _Znam+0x74 (libclang_rt.asan_osx_dynamic.dylib:arm64e
    #1 0x000100424838 in g(double)+0x20 (stack-trace:arm64+0x100000838)
```

An example

```cpp
#include <iostream>

double g(double b) {
    double* v = new double[5];
    v[0] = 0;
    for (int i = 1; i != 10; ++i) v[i] = v[i - 1] + b * i;
    double ret = v[4];
    delete[] v;
    return ret;
}

double f(int a) {
    static const double PI = 3.14;
    double b = a * PI;
    return g(b);
}

int main() {
    int x = 3;
    std::cout << f(x) << std::endl;
    return 0;
}
```

Q. Where is the problem in this piece of code?

With option `-fno-omit-frame-pointer -g`

```
[→  sanitizers-example-project git:(25-26) ✗ ./build/stack-trace
=================================================================
==2728==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x6040000003
WRITE of size 8 at 0x6040000003b8 thread T0
    #0 0x000104c50930 in g(double) stack-trace.cpp:6
    #1 0x000104c50a04 in f(int) stack-trace.cpp:15
    #2 0x000104c50a38 in main stack-trace.cpp:20
    #3 0x0001896c9d50  (<unknown module>)

0x6040000003b8 is located 0 bytes after 40-byte region [0x604000000390,0x6040
allocated by thread T0 here:
    #0 0x00010546b548 in _Znam+0x74 (libclang_rt.asan_osx_dynamic.dylib:arm64
    #1 0x000104c5083c in g(double) stack-trace.cpp:4
    #2 0x000104c50a04 in f(int) stack-trace.cpp:15
    #3 0x000104c50a38 in main stack-trace.cpp:20
    #4 0x0001896c9d50  (<unknown module>)
```

Understanding the flag `no-omit-frame-pointer`

- Don’t confuse the return address (where the function should go back to) with the address stored in the frame pointer!
- The return-address is the address of the **next code instruction** to execute when the function returns, to keep executing.
- The frame pointer keeps the **address of the data** (on the stack) manipulated by the function.